

Day-1

C++

Page No.:

Date: / /

2 types of lang:- ① High level

- where instructions are written in english
- machine doesn't understand high level lang.
- compiler is used to convert high-level code to low-level code.
- C++ is a high-level language.

★ Procedure Oriented Programming :- C, Basic

★ Object Oriented language :- C++, Java,

★ Object Based language :- VB, JavaScript
(In this inheritance is not supported fully)

★ functional Based language :- Lisp, python

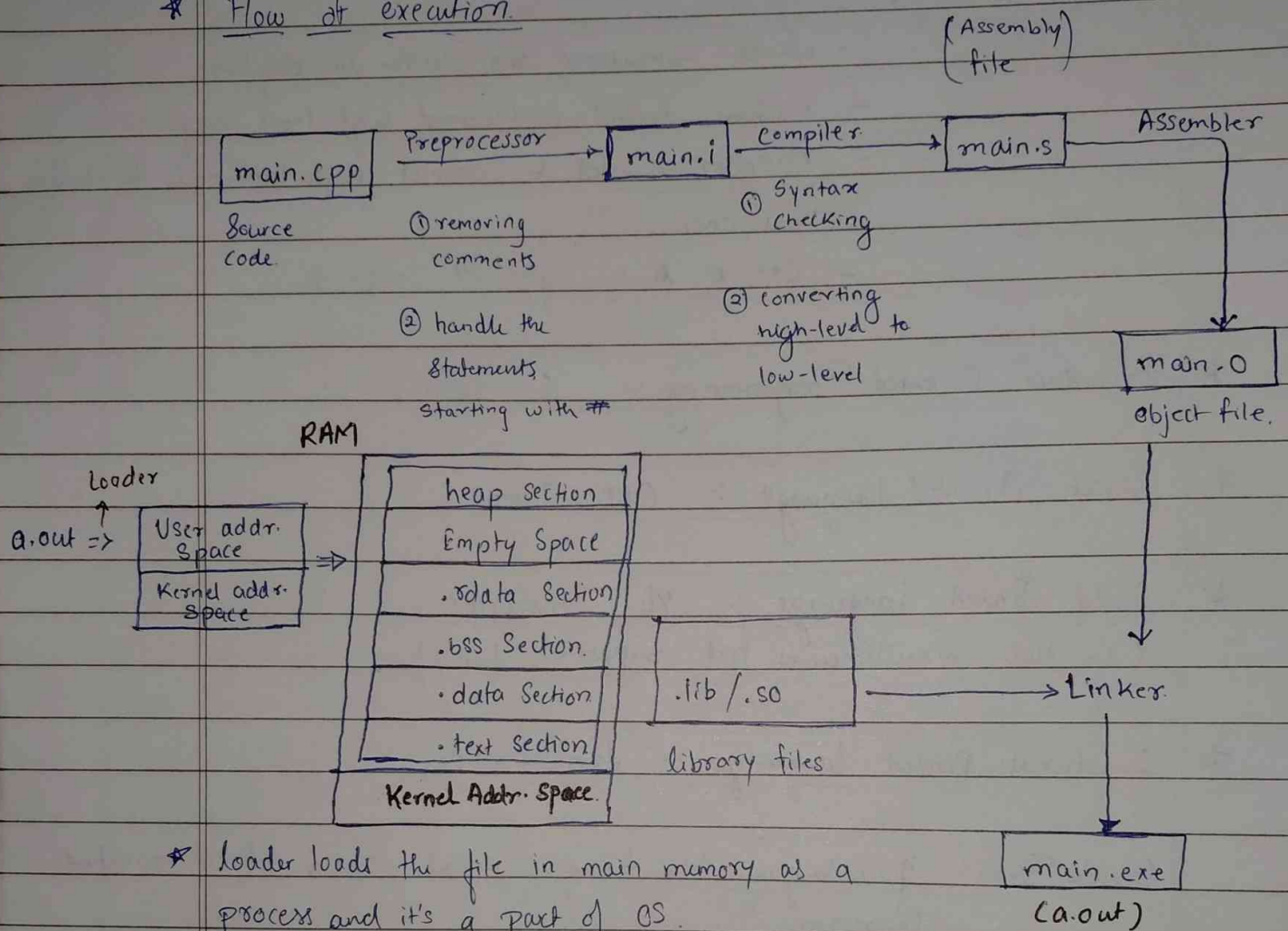
Grady Buch:- if a language has 4 major pillars, its a object oriented language

- ① Abstraction :- To achieve simplicity
- ② Encapsulation :- To achieve data hiding
- ③ Modularity :- To minimize module dependency
- ④ Hierarchy :- To achieve reusability

3 - Minor Pillars :-

- ① Typing :- To reduce maintainence of system
- ② Concurrency :- To perform multiple tasks simultaneously.
- ③ Persistence :- To store state of object on secondary storage.

* Flow of execution



* Loader loads the file in main memory as a process and it's a part of OS.

* if we don't write main in code, it'll not give a compiler error, it will give a linker error.

- ld in error stands for dynamic linker.

* compilation starts from line 1 & execution starts from main().

* Data type of any variable decides 4 things

- ① Memory
- ② Range
- ③ Operation
- ④ Nature

* local declarations has by default garbage values & global declarations has 0 in it.

Page No.:
Date: / /

data types

① Fundamental	② Derived	③ User-Defined
- int (4) - float (4) - char (1) - double (8) - bool (1) - wchar_t (2) - void (not mentioned)	- Array - Function - Pointer - Reference	- Enum - Struct. - Class - Union.

*

```
struct Employee {  
    - - - - -  
    - - -  
};
```

Type:- struct Employee

variable:- e1

Declare a member:-

```
struct Employee e1 = {"Ketan", 101, 1200.00f};
```

* In C, struct's global functions were able to change struct objects, i.e. unidentif-ied access. So, to counter, declare functions in struct (inside) in C++.

* class is a basic unit of encapsulation.

* Only diff b/w class & struct in C++ is, by default members in struct are public, ~~but~~ and by default members in class are private.

- ★ Only ~~an~~ object can be created from a concrete class and not from an abstract class.

Page No.:	
Date:	/ /

→ In an object, only non-static data members get memory

- ★ Instantiation means, creating an object of a class.

- ★ Calling a member function ~~of~~ using an object, it's called message passing.

Size of object = size of all non-static data members of a class.

- ★ A method with body is concrete method and without body is abstract method. (method = member function)

Special.

- ★ Compiler gives us 4 ^{special} member functions, under certain scenarios,

① Constructor

② Destructor

③ Assignment Operator Function

④ Copy Constructor.

- ★

```
const int num = 10;  
const int *ptr = &num;
```

here ptr is not a constant, it's just a representation, that it stores address of a constant number/integer

if const comes before *, the value is constant, not the pointer.

- ★ Storage classes in C/C++ :- ① auto ② static ③ extern ④ register

- ★ Storage class ~~defines~~ scope of variable and function and lifetime of the variable.
decides

* this is a keyword (that stores object address)

Page No.:

Date: / /

```
* int num = 10
  int * const ptr = &num
```

Here, the pointer is constant and the num is a regular variable.

As, * comes before const, so our ptr stores a address of num & can't be modified.

* If a pointer is constant, it needs to be initialized on the same line.

* Now, if both value & pointer are const, it can be declared as:-

```
const int num = 10
const int * const ptr = &num;
```

(We need to read from left to right, i.e., ptr is a constant pointer storing address of constant integer).

* Scope

- Area / region / boundary where we can access variable / function.

```
namespace na
{ int num3 = 30 // namespace scope.
  class C1 {
    int num4; // class scope
  }
}
```

* this is a keyword (that stores object address)

Page No.:	
Date:	/ /

```
int num = 10
int * const ptr = &num
```

Here, the pointer is constant and the num is a regular variable.

As, * comes before const, so our ptr stores a address of num & can't be modified.

* If a pointer is constant, it needs to be initialized on the same line.

* Now, if both value & pointer are const, it can be declared as:-

```
const int num = 10
const int * const ptr = &num;
```

(We need to read from left to right, i.e., ptr is a constant pointer storing address of constant integer).

* Scope

- Area / region / boundary where we can access variable / function.

```
namespace na
{ int num3 = 30 // namespace scope.
  class C1 {
    int num4; // class scope
  }
}
```

- * `::` Variable_name to access global scope variables having same name.

Page No.:	
Date:	/ /

```
int main ()
{
    int num5 = 10; // Functional Scope
    {
        int num6 = 10 // Block Scope
    }
}
```

* Namespaces:-

→ To group Functionally & Logically related types, we use namespace.

Day -2

- * To avoid name clashing, we use namespaces.

→ Namespaces are to be declared globally, their names are in small case letters.

- * Convention for macro and constant

```
#define NULL 0
#define EOF -1
```

* Namespace

Ex:- namespace na

```
{
    int num1 = 10;
}
```

```
int main ()
{
```

```
    printf ("num1: %.d", na::num1);
```

```
}
```

access using scope resolution operator

* If namespaces ~~are~~ name are same, then names of members should be different and vice-versa

* Namespaces can be nested. (For access :- `ns1::ns2::name1`)

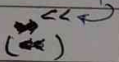
* To counter multiple scope resolution operator for multiple nested namespaces, so, we use "using namespace namespace-name".

→ "using" means it's making a request to compiler, so, the priority will be given to local scope firstly.

* We can write Functions and Structs ^{and classes} inside a namespace.

→ We can't keep main inside namespace, as it's a entry point function.

(Insertion operator)



* cout → character output (typedef basic_ostream<char> ostream;)



* cin → character input

(Extraction operator)

* endl, set-precision, all manipulators

* Steps to write a class

① Understand requirement and Declare the class.

② ~~Decl~~ Declare data members / fields

③ Creating the object.

④ Access Specifier.

⑤ Calling methods.

* "using" is a directive, which we can use to frequently access members of namespace.

* Characteristics of Object

① State ② Behavior ③ Identity

- State

- it represents data/values present inside the object.

Employee emp1;

name	"Om"
empId	1
Salary	1000.00

- Behavior

- calling member functions on a object decides the behavior of an object.
- member function defined inside class ~~defines~~ represents behavior of object.

Ex:- emp1.acceptRecord();
 emp1.printRecord();

- Identity

- a unique field present inside an object. (for ex:- empId, etc). that distinguishes it from all other objects.

* Member Functions do not get space inside object, rather all objects of same class, share a single copy of it.

* main & destructor can't be overloaded

* Function overloading can't be done on return type.

($\because$ catching value from a function is - optional)

* Function Overloading

→ Compile-time polymorphism

→ implementation of function is logically same then we can give same name to the function.

→ If same name, either the types or the number of ^{parameters} ~~polymorphism~~ should be different, also order of parameters can be different, if types & number of params are same.

* To identify ~~same~~ which function to call, compiler uses name mangling to create mangled names.

for ex:- `void Sum (int num1, float num2) // zsumif`

`void Sum (float num1, double num2) // zsumfd`

* this pointer

→ it is a constant pointer

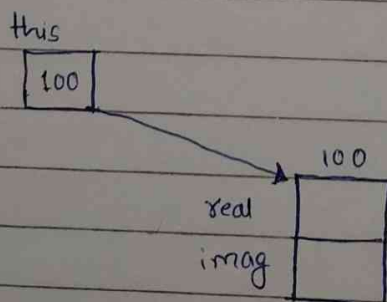
→ carries the address of the object.

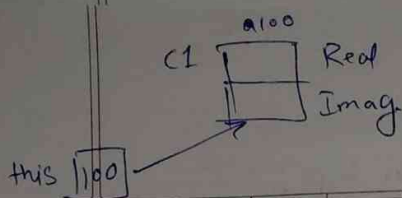
Ex:- `Complex c1; // @ init`

`c1.printComplex`

`c1.initComplex // c1.initComplex(&c1);`

i.e.





C1.initComplex (& c1)

Page No.:	
Date:	/ /

★ In order to access or set private members of class, we can use setter & getter functions.

```
void setReal (int Real) // setter function
{
    this -> real = real;
}
```

```
int void getReal () // getter Function
{
    return real;
}
```

★ we use "this" compulsarily when the names of param. of member function and name of data members of class is same.

★ General type of this pointer :- className * const this;

★ Global, static member & friend function doesn't get the "this" pointer.

★ Member Functions of a class are by default inline.

* Adding 2 objects, means addition of states of objects.

* this - pointer example

```
class Complex {
```

```
    int real;
```

```
    int imag;
```

```
    public:
```

```
    Complex Sum (Complex other)
```

```
    {    Complex temp;
```

```
        temp.real = this->real + other.real;
```

```
        temp.imag = this->imag + other.imag;
```

```
        return temp;
```

```
    }
```

```
};
```

```
int main ()
```

```
{    Complex c1;
```

```
    c1.acceptRecord (); // 10 20
```

```
    Complex c2;
```

```
    c2.acceptRecord (); // 30 40
```

```
    Complex c3;
```

```
    c3 = c1.Sum (c2); // c1.Sum (&c1, c2) → c3
```

```
    return 0;
```

```
}
```

other

80	real
40	imag

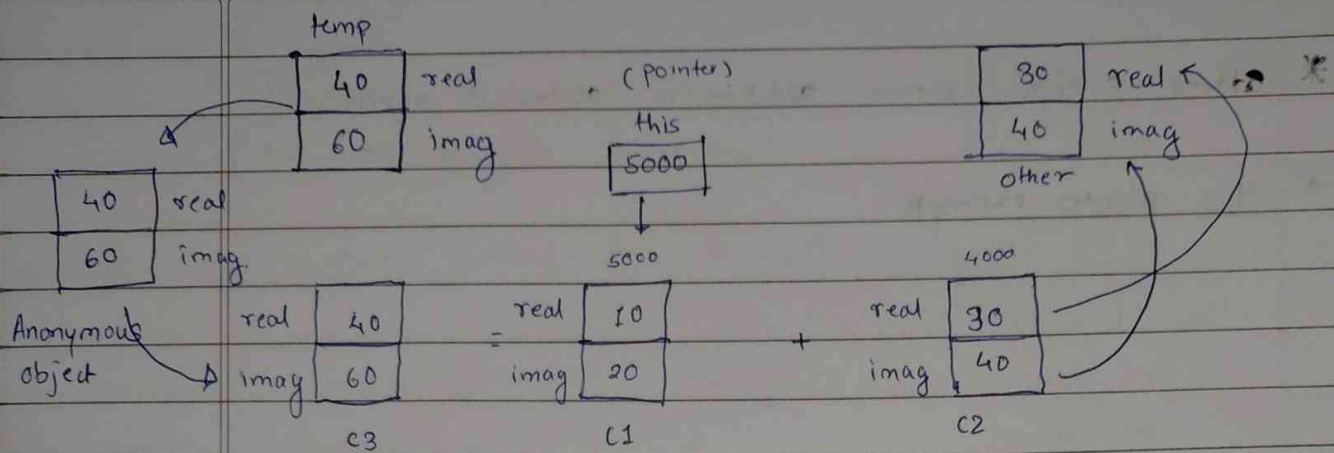
100

c1	10	real
	20	imag

200

c2	30	real
	40	imag

c3	40	real
	60	imag



* Constructor

- Constructor is a special member function, whose purpose is to initialize an object of class.

Rules:-

- ① Name of class == Name of constructor
- ② It doesn't have a return type
- ③ It can accept parameters.

- Constructor is called when an object is created and not on a pointer or reference.
- Constructor cannot be declared static, constant, volatile, virtual.

Complex & c3 = c2; // Constructor won't be called on c3, as it's a "reference".

- Constructor is called o/c to the order of declaration of objects.
- If constructor is public, we can create objects outside/inside the class &

- If constructor is private, we can create objects only inside the class. (in Singleton class)
- So if we need to create object outside class & constructor is private, we can
 - ① declare a static function inside class.
 - ② Create an object inside static function.
 - ③ then call that static function from outside class.
- 3 types of constructor :- ① Parameterless
② Parameterised
③ Default
- Copy constructor is not a type of constructor.

// function declaration (constructor not called)
Complex C1 ();

// Complex C2 (10) (single parameterized ctor called)
Complex C2 = 10;

// Anonymous Object. → member function can be called (Complex(10, 20). print());
Complex (10, 20); // 2-argument ctor called.

(C++11) & Complex C4 = { 10, 20 } // OK, 2-argument ctor called

Feature

Complex C5 = (10, 20) // single-parameter ctor called & rightmost
↓ more like value is given priority.

Complex C5 = 20 ;

Teacher's Signature:

★ Initialization is done only once & assignment can be done multiple times.

Page No.:	
Date:	/ /

(Imp)

* Constructor Delegation (Also used in Java, called constructor chaining)

- calling another constructor from one constructor.

```
Complex (void) : Complex (0, 0)
{
    cout << "Parameterless" << endl;
}
```

```
Complex (int real, int imag)
{
    this -> real = real;
    this -> imag = imag;
}
```

So, object creation:-

```
Complex c1;
```

X Here, firstly, parameterless constructor will be called and then 2-argument constructor will be called.;

Parameterless → 2-Argument ctor
(cout will be executed) → (this → real = real &
this → imag = imag)

* Constructor member initialization list

→ By default, compiler doesn't follow the order of data members, it follows the order of constructor, i.e.

```
Class Test {
    Private: int num1;
            int num2;
```

★ String or Array can't be initialized inside ~~data~~ member initializer list.
constructor

Page No.:	
Date:	/ /

Public:

Test (void)

```
{  
    this → num3 = num2;  
    this → num2 = num1;  
    this → num1 = 0;  
}
```

};

But, if we want to follow order of data members, we can write constructor as,

```
Test (void) : num3 (num2) , num2 (num1) , num1 (0) ;  
{  
}
```

(Here, firstly num1 will get initialized, then num2 & then num3)
for parameterized ctor:-

```
Test (int n1 num1, int n2 num2) : num1 (n1) , num2 (n2)  
{  
}
```

PTO

- data members can be made constant inside a class.

* Const

- when a data member is constant, it needs to be initialized using the constructor member initializer list.

```
class Test {  
    private:  
        const int num;  
    public:  
        Test (void) : num (0)  
        {  
        }  
};
```

- Facilitator Functions (like print, etc), that prints the state of object can be made constant.

```
class Test  
{  
    const  
    int num1;  
  
    Test : num1 (0) {}  
  
    public:  
        void showRecord () const  
        {  
            ++, this → num1; →  
        }  
    cout << num1  
}
```

Not Allowed. because member function is constant.
So, it makes the current object constant, i.e. it cannot be modified.
(Imp)

- Date: / /
- ★ To be able to modify any data member ~~fun~~ inside a constant member function, we should make it mutable.

i.e. Public:

mutable int count;

- ★ Constant objects can only call constant member functions.

★ Reference

- used to hide the complexity of pointer.
- used to create alias of existing object (not for constant literal)
- we cannot keep reference uninitialized

main ()

{

int num1 = 10; (Referent)

int &num2 = num1; (Reference)

return 0;

}

ex:- ++num1; // 11

++num2; // 12

- Reference is internally a constant pointer.

int num1 = 10

int num2 = 20

int &num3 = num1;

num3 = num2;

→ here the reference of num1[↑] will be set its value to 20. i.e. num3

★ References ^{are} always initialized in a constructor member initializer list.

Page No.:

Date: / /

```
class Test
{ private:
    char &ref;
public:
    Test (char &ch) : ref(ch)
    { }

    void incr ()
    { ref++
    }

    void display ()
    {
        cout << ref << endl;
    }
};
```

main ()

```
{ Test ch1;
```

```
    char ch1 = 'A';
```

```
    char &ch2 = ch1;
```

```
    cout << sizeof (ch2); // 1
```

```
    cout << sizeof (Test); // 4
```

```
    char ch3 = 'A';
```

```
    Test t1 (ch3);
```

```
    t1.incr ();
```

```
    t1.display (); // 'B'
```

```
    char ch1 = 'A', ch2 = 'B', ch3 = 'C';
```

```
    char &arr [3] = { ch1, ch2, ch3 } → NOT ALLOWED
```


// instead, we do

```
Test arr[3] = { ch1, ch2, ch3 }
```

```
for (i=0 → 3)
```

```
arr[i].incr()
```

```
for (i=0 → 3)
```

```
arr[i].display() // "B" "C" "D"
```

```
}
```

* Dynamic Memory Allocation

- Allocating memory at runtime.
- memory allocated through malloc contains garbage
- memory allocated through calloc contains '0' (zero).
- realloc is used to resize the memory.

→ void pointer can't be dereferenced.

Return type of malloc is :-

→ typedef of unsigned integer

void * malloc (size_t size)

* malloc

```
main()
```

```
{ int * ptr;
```

```
    malloc
```

```
    ptr = (int *) malloc (sizeof(int));
```

```
    if (ptr != NULL)
```

```
    { *ptr = 200;
```

```
      cout << *ptr << endl;
```

```
      free (ptr);
```

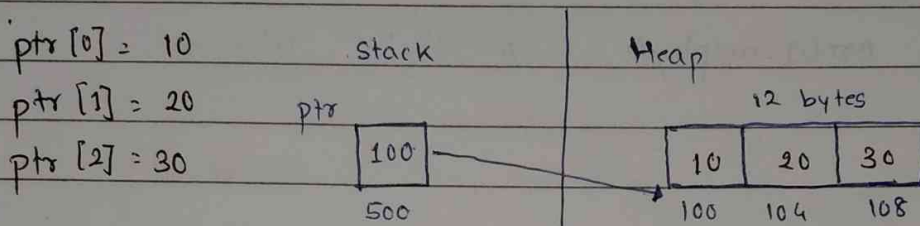
```
      ptr = NULL;
```

```
}
```

★ Pointer containing an address which is already freed/unavailable, is called Dangling Pointer.

- Malloc returns NULL if failed to allocate memory.
- new throws an exception (bad_alloc) if failed to allocate memory

★ `int * ptr;`
`ptr = (int *) malloc (3 * sizeof (int));`



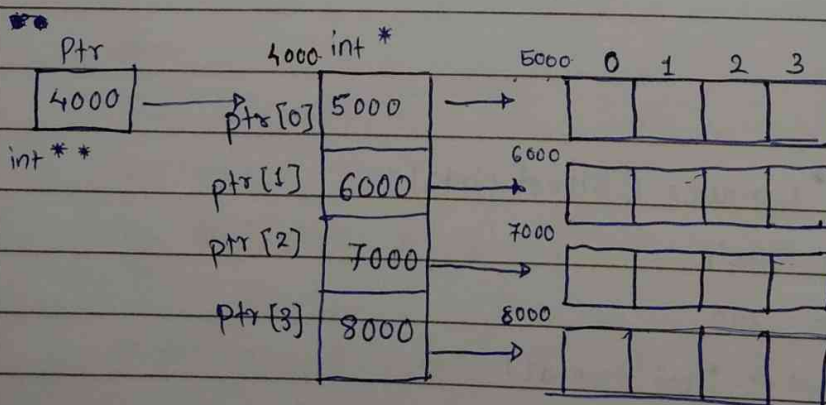
`ptr[0] = *(ptr + 0) = 10`

`ptr[1] = *(ptr + 1 * 4)`
`= *(104) = 20`

★ Calloc

`int * ptr = (int *) calloc (1, sizeof (int));`

DMA (2D Array)



(Imp)

Code:-

```
int ** ptr = (int **) calloc (4, sizeof (int *));
```

```
for (i=0 → 4)
```

```
ptr[i] = (int *) calloc (4, sizeof (int));
```

```
for (i=0 → 4)
```

```
for (j=0 → 4)
```

```
cin >> ptr[i][j];
```

} input.

```
for (i=0 → 4)
```

```
for (j=0 → 4)
```

```
cout << ptr[i][j];
```

```
cout << endl;
```

```
for (i=0 → 4)
```

```
free (ptr[i]);
```

```
ptr[i] = NULL;
```

// freeing each row allocated in heap

```
free (ptr);
```

```
ptr = NULL;
```

// freeing array of pointers.

P.T.O

- ★ When allocating memory using malloc, constructor doesn't get called.

ex:-

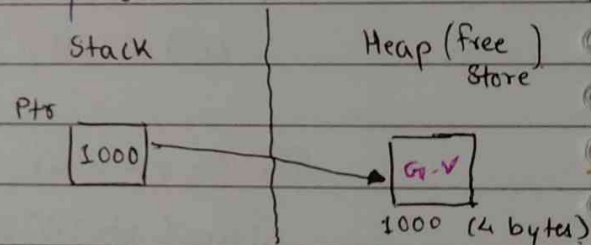
```
Complex * ptr = (Complex *) malloc (sizeof (Complex));
// no constructor called
ptr -> acceptRecord();
ptr -> printRecord();
free (ptr);
ptr = NULL;
```

★ Dynamic Memory Allocation in C++

- malloc is a function, whereas new is a operator.
- Free Store is the name of heap memory in C++.

using new:-

```
int * ptr = new int;
```



- memory allocated in heap memory doesn't have names. they are anonymous.

using delete:-

delete ptr; → To avoid memory leakage

then,

```
ptr = NULL;
```

- ★ initialization syntaxes for new.

PTO

Heap, Stack, Data segment $\Rightarrow$ Free Store

Page No.:
Date:

① `int * ptr = new int;`

Stack: ptr [5000] (int*)

Heap: [G.V] (int)

② `int * ptr = new int(1);`

Stack: ptr [6000] (int*)

Heap: [0] (int)

③ `int * ptr = new int(3);`

Stack: ptr [7000] (int*)

Heap: [3] (int)

★ Allocating memory for multiple values / array.

`int * ptr = new int[3];`

to access/set:- `ptr[0]=10, ptr[1]=20, ptr[2]=30;`

Stack: ptr [100] (int*)

Heap: [10 | 20 | 30] (int)

Indices: 0 1 2

to delete:-

`delete [] ptr;`
`ptr = NULL;`

★ Allocating heap memory to 2D array.

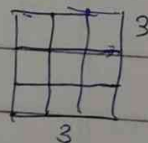
`int ** ptr = new int*[3];`

$\nearrow$ ptr to array of pointers

for (i=0 $\rightarrow$ 3)

`ptr[i] = new int[3];`

$\rightarrow$ this will create an array of size (3x3)



★ new returns / throws an exception (bad_alloc) which is a runtime error.

* Consider size of string class = 24

Page No.:

Date: / /

Reference vs Pointer

① We can create a pointer-to-pointer whereas no concept of reference-to-reference.

(Dereference) ② We need to explicitly dereference a pointer, whereas a reference is dereferenced automatically.

(Initialization) ③ Pointer can be initialized to NULL; whereas reference can't be initialized to NULL.

(init once) ④ Reference can be initialized only once, unlike pointer ~~which~~ which can be initialized multiple times.

* Class using dynamic memory allocation.

Class Student

```
{  
    int marks;  
    double marks;  
    string name;  
    int Roll;  
}
```

Public:

Student () : marks (0) , name ("") , roll (0) {}

Student (marks, name, roll) : marks(marks), name(name), roll(roll) {}

void accept () { --- }

void display () { --- }

};

①

main()

{

Student *arr[3];

arr[0] = { 33.0, "Om", 1 };

arr[1] = { 44.0, "Omi", 2 };

arr[2] = { 55.0, "Omni", 3 };

// to free memory

for (i=0 → 3)

delete arr[i];

arr[i] = NULL;

}

②

// using new
~~using new~~

main()

{

Student *arr[3] = new Student[3];

for (i=0 → 3)

arr[i].accept();

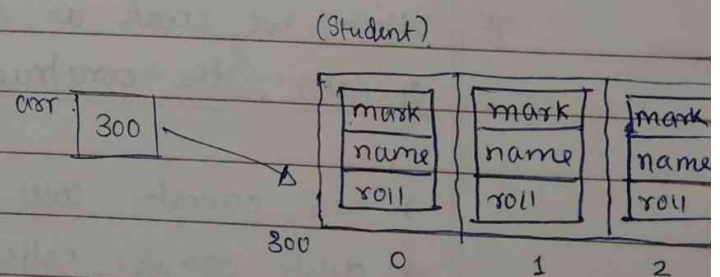
for (i=0 → 3)

arr[i].print();

delete[] arr;

arr = NULL;

}



③

main()

{

Student **arr = new *Student[2]

arr[0] = new Student (12.2, "Om", 1);

arr[1] = new Student (14.4, "omi", 2);

for (i=0 → 2)

arr[i] → display();

for (i=0 → 2)

~~del~~

delete [] arr[i];

arr[i] = NULL;

delete arr;

arr = NULL;

}

★ When we create an object in dynamic memory using new operator, the constructor is called.

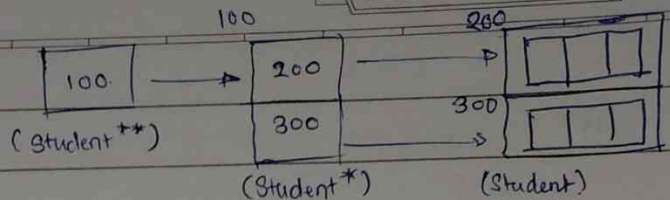
→ new operator calls constructor.

→ delete operator calls destructor

~Student () {

cout << "called" << endl;

}



* Array class

Class Array

{

int size;

int * arr;

public:

Array (int size) {

this->size = size

this->arr = new int [this->size];

}

void acceptRecord ()

{

for (i=0 -> this->size)

~~arr[i]~~

cin >> arr[i];

}

void printRecord ()

{

for (i=0 -> this->size)

cout << arr[i];

}

void FreeArr ()

{

if (this->arr != NULL)

delete [] arr;

arr = NULL;

}

} // But here developer can call FreeArr multiple times, which is not proper. So we use a destructor.

So, we'll write

```
~Array ()           // instead of free Arr
{
    if (this->arr != NULL)
    {
        delete [] arr;
        arr = NULL;
    }
}
```

Destructor :-

- ① Same name as class & precedes with (~)
- ② Doesn't have a return type.
- ③ Doesn't accept any parameters.
- ④ Don't need to be called implicitly.

*** Also, why destructor can't be overloaded?

→ As it doesn't take any parameters, so we can't overload.

→ Destructor is a member function of a class, which is used to release resources held by the object.

→ Destructor releases the resources of last object first then its prev. basically, LIFO.

Static

data Section

- * Static numbers get memory ^{only once} before execution of program in data section, & it's initialized to zero (0).

```
void print ()  
{  
    static int num = 10;  
    num++;  
    cout << num << endl;  
}
```

```
main ()  
{  
    print(); // 11  
    print(); // 12  
    print(); // 13  
}
```

// uninitialized data variables gets memory in .bss section.
(block started by system)

- * Static data numbers gets memory once per class.

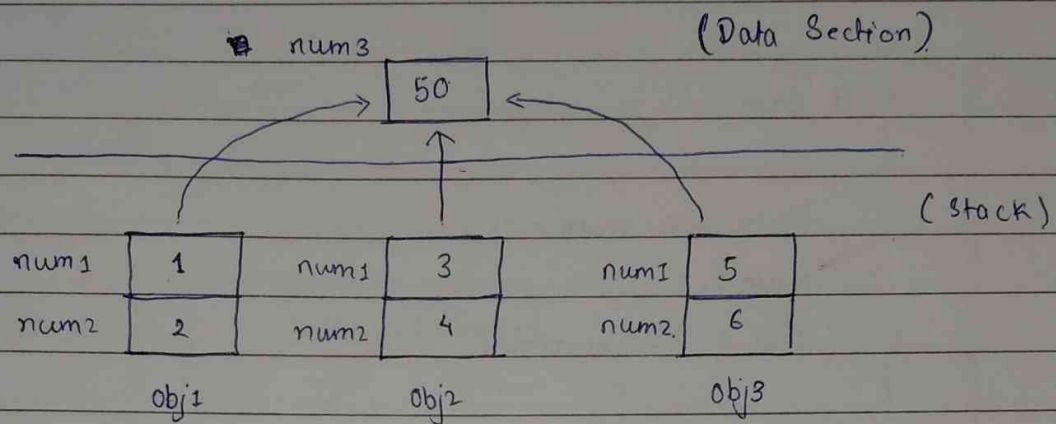
Static data members should not be initialized inside a constructor. Instead, we should provide its global declaration.

```
int Test :: num3 = 50; // global declaration
```

- * non-static data numbers gets memory once per object.

* Static data member is a class-level variable.

* Static data members are accessed using class name.



* For static data members, we need to create static member functions.

i.e.

```
class Test
```

```
{ int num1;
```

```
  static int num2;
```

```
public:
```

```
  void SetNum1 ( int num1)
```

```
{
```

```
    this->num1 = num1;
```

```
}
```

```
  static void SetNum2 ( int num2)
```

```
{
```

```
    Test :: num2 = num2;
```

```
}
```

```
};
```

```
int Test :: num2 = 0;
```

// global definition
// global declaration

* Static member functions are to be called using classes.

* We cannot declare static function :-

- ① Constant
- ② Virtual
- ③ Volatile

* Where we do not process the state of an object inside the function, we can make that function static.

* We need to give the global definition of static data member every time we create a static data member.

★ Anonymous Class

→ a class without a name.

→ exception: here only we can call static member function using object.

```
class {  
    public:  
    static void show ()  
    {  
        - - - -  
    }  
  
    static void accept ()  
    {  
        - - -  
    }  
} c1;
```

in main:-

```
c1.show();  
c1.accept();
```



* Abstraction

- abstraction focuses on the essential characteristics of some object, relative to the perspective of the viewer.
- it's a process of getting essential things from object.
- using abstraction we can achieve simplicity.
- describes outer behaviour of object.

* Encapsulation:-

- Binding of data and code together is called encapsulation.
- it represents internal behaviour of object.
- it is used to data hiding.

* Modularity

- Process of developing complex systems using small parts.
- Using modularity, we can reduce module dependency.

* Hierarchy

- Level / Order / Ranking of a abstraction is called hierarchy.
- Main purpose of hierarchy is to achieve reusability.

Advantages:-

- ① reduce development time
- ② reduce development cost
- ③ reduce developer effort

Types of hierarchy:-

- ① Has-a / Part-of $\Rightarrow$ Association
- ② Is-a / Kind-of $\Rightarrow$ Inheritance / Generalization
- ③ Use-a $\Rightarrow$ Dependency
- ④ Creator-a $\Rightarrow$ Instantiation.

Association:- (Has-a relationship)

2 types $\rightarrow$ ① Composition. (tight coupling)

② Aggregation (loose coupling)

* If dependency object exists without dependant object, it is called aggregation.
Hence, also called loose coupling.

* If dependency object do not exists without dependant object, it is called composition.
Hence, also called tight coupling.

*** if a object gets space inside another object its called tight coupling.

Page No.:	
Date:	/ /



```
Class Date
{
    - - -
    - - -
}
```

(composition)

```
Class Employee
```

```
{
    int empid;
    int salary;
    Date joinDate; // Association
public:
```

```
Employee (empid, salary, joinDate) : empid(empid), salary(salary),
                                     joinDate(joinDate) {}
```

```
void print()
```

```
{
    cout << empid << salary << this->joinDate.printDate();
    cout << endl;
}
```

// we can also do

```
Employee (empid, salary, day, month, year) : empid(empid), salary(salary),
                                             joinDate(day, month, year)
```

```
void PrintEmp ()
{
```

```
    cout << empid << salary << this->joinDate.printDate;
```

```
};
```

* Class Customer (loose Coupling Ex) (Aggregation)

```
{
    int id;
    string name;
    Date *dob;
    public:
```

```
    Customer () : id(0), name("") { dob = NULL }
```

```
    Customer (id, name) : id(id), name(name) { dob = NULL }
```

```
    Customer (id, name, day, month, year) : Customer (id, name)
    {
        dob = new Date (day, month, year);
    }
```

```
    void acceptDate ()
    {
```

```
        dob = new Date ();
```

```
        dob -> acceptDate ();
```

```
    }
```

```
    void accept (Customer c)
```

```
    {
```

```
        cin >> id >> name;
```

```
    }
```

```
    void print Customer ()
```

```
    {
```

```
        cout << id << name << endl;
```

```
        if ( dob != NULL )
```

```
            dob -> printDate ();
```

```
    }
```

```

~ Customer ()
{
    if (dob != NULL)
    {
        delete dob;
        dob = NULL;
    }
}

```

Since here, we are ^{having} ~~storing~~ a pointer (dob), which is storing a pointer to Date object. That's why, the object does not get space inside customer object. So, this is called loose coupling.

* Inheritance (Generalization)

```

class Employee  class Person (Base class)
    ↓
class Employee (Derived class)

```

- Default mode of inheritance is private.

```

class Person : Public Employee.

```

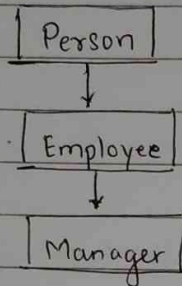
Inheritance
mode

{ Public, Private, Protected }

- * Private members of ^{Person} ~~Employee~~ class gets inherited, but can't be accessed.

Size of Derived class object = $\overset{\text{all}}{\uparrow}$ Size of non-static data members of base class + $\overset{\text{all}}{\uparrow}$ Size of non-static data members derived class.

- * Static data members gets inherited in derived class from base class also static ~~data~~ member function gets inherited, and are accessible in derived class.



- * Employee is direct derived class of Person.
- * Manager is indirect derived class of Person.
- * Person is direct base class of Employee.
- * Person is indirect base class of ~~Base~~ Manager.

- * Use private mode of inheritance, if it is a "has-a" relationship.
- * Use public mode of inheritance, if it's a "is-a" relationship.
- (Imp) ** If creating an object of derived class (without parameters), the constructor of base class gets called first and then of derived class.
- ** And, destructor calling sequence is opposite so, first destructor of derived class gets called first & then destructor of ~~derived~~ ^{base} class gets called.
- ** From any constructor of derived class, base class's parameterless constructor gets called.
- ** Using constructor base initializer list, we can call parameterized constructor of base class from derived class.
- ** Employee (name, age, empid, salary): empid(empid), salary(salary),
Person(name, age).

Teacher's Signature:

* We need to write parameterless constructor in base class compulsorily.

Page No.:	
Date:	/ /

Day - 5

* If there exist same name function in base & derived class, then calling that member function on derived class object, it will call the derived class member function. This concept is called Shadowing.
(as it hides the implementation of base class member function).

So, we can use scope resolution operator to call member function of base class.
i.e.

Base::member_function();

OR

this → Base::member_function;

* Types of Inheritance

① Implementation Inheritance

→ Inheritance happening b/w two classes, then it's a Implementation Inheritance.

- Types of implementation inheritance

① Single

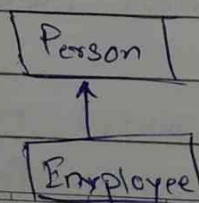
② Multiple

③ Multilevel

④ Hierarchical

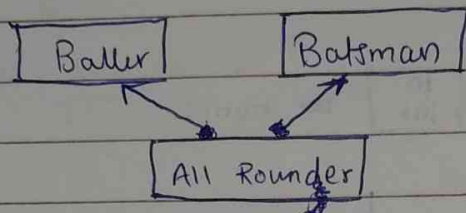
① Single Inheritance

A single base class having a single derived class.



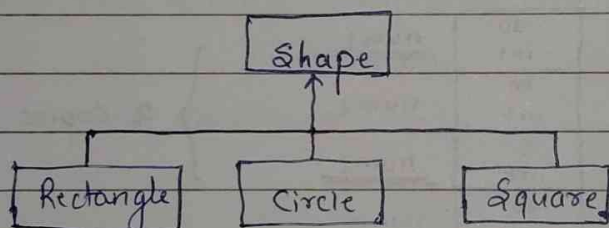
(b) Multiple Inheritance

If a class is having multiple base classes & single derived ~~base~~ class, then it's called multiple inheritance.



(c) Hierarchical Inheritance

If single base class is having multiple derived class, then it's called hierarchical inheritance.



(d) Multi-level Inheritance

If single inheritance is having multiple levels then it is called multi-level inheritance.

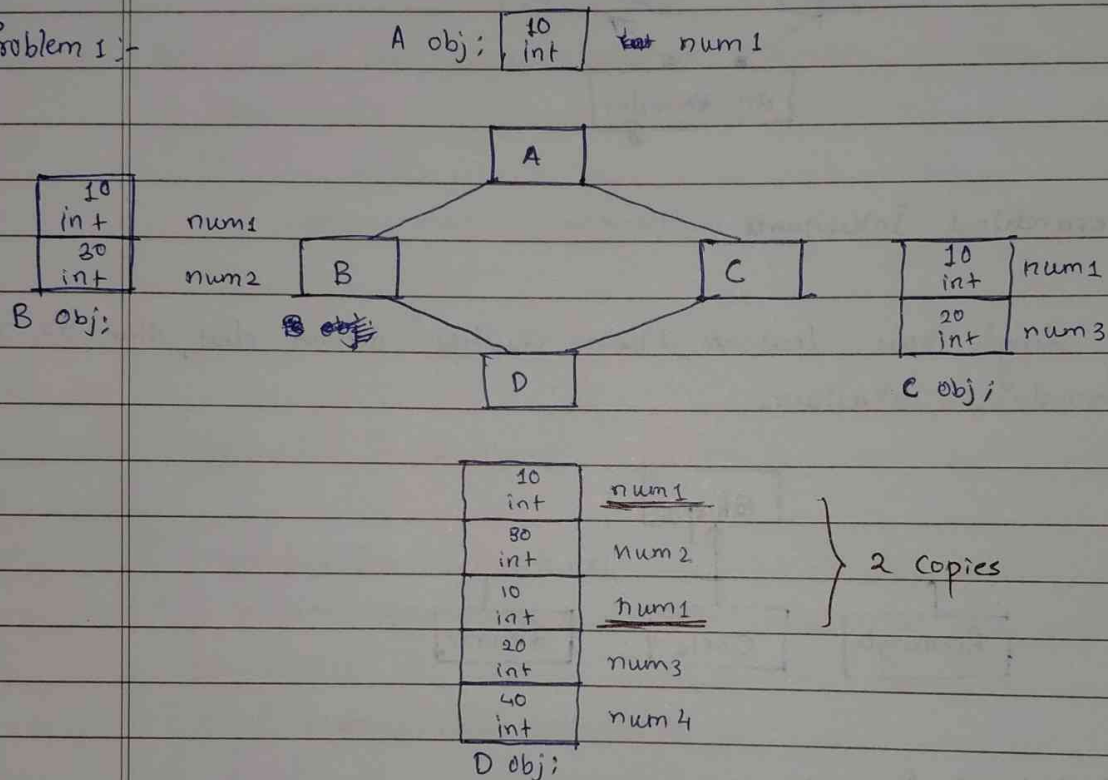


Teacher's Signature:.....

types of
 ★ Hybrid Inheritance:- Combination of many inheritance i.e.
 (Single, multiple, multilevel, hierarchical)

Imp * Diamond Problem

Problem 1:-



→ Here in D class, when we create a object, the object will have 2 copies of num1. (This is the problem).

→ Size of object is increasing & this will become a problem once no. of data members are ~~more~~ ↑ge.

★ In case of diamond inheritance, member functions of indirect base class, inherit into indirect derived class multiple times.

★ If we try to call member function of indirect base class on object of indirect derived class, then compiler generates ambiguity errors.

Teacher's Signature:

Problem 2:-

Another problem arises, when we try to call member function of class A from class D. As, it has multiple choices to call from either B or C.

void showRecord ();

A

void A::showRecord ();

void A::showRecord ();

void showRecord (); **B**

C

void showRecord ();

D

void B::A::showRecord (); ①

void B::showRecord ();

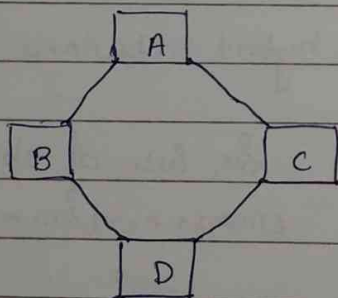
void C::A::showRecord (); ②

void C::showRecord ();

void showRecord ();

} gets called twice.

Problem 3:-



A Class Object

- constructor call : A()
- Destructor call : ~A()

B Class Object

- constructor call : B()
- Destructor call : ~B()

C Class object

- constructor call : C()
- Destructor call : ~C()

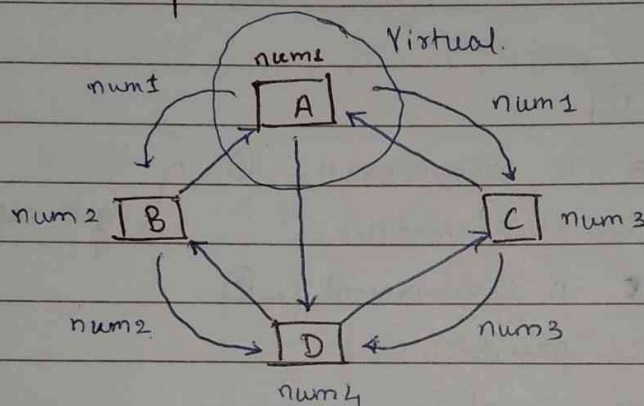
D class object :-

Constructor call $\rightarrow A() \rightarrow B() \rightarrow A() \rightarrow C() \rightarrow D()$

Destructor call $\rightarrow \sim D() \rightarrow \sim C() \rightarrow \sim A() \rightarrow \sim B() \rightarrow \sim A()$

problem is :- constructor & destructor of class A gets called twice.

(Imp) * Solution to problem is :-



Diamond Problem = Hierarchical Inheritance + Multiple Inheritance.

- All these problems are created by hybrid inheritance.

num1	40 int
num2	20 int
(B)	vbptr
num3	30 int
(C)	vbptr
num4	40 int

So, size of obj of D is
 $(num1 + num2 + num3 + num4)$
 $+$
 $(vbptr + vbptr)$
 $= (16) + (8)$
 $= \underline{24} \text{ bytes.}$

vbptr $\rightarrow$ pointer to virtual base class object

- * If you want to avoid diamond problem, we need to declare base class virtual, i.e. we should derive class B & C virtually from A (base) class.

```
class B : virtual public A
class C : virtual public A } virtual inheritance
class D : virtual A
```

* Upcasting and Downcasting

```
class Base
{
    int num1, num2;
public:
    Base () : num1(0), num2(0) {}
    Base (num1, num2) : num1(num1), num2(num2) {}

    void ShowRecord () {
        cout << num1 << num2 << endl;
    }

    void printRecord () {
        cout << num1 << num2 << endl;
    }
}
```

```
class Derived : public Base
{
    int num3;
public:
    Derived () : num3(0) {}
    Derived (num1, num2, num3) : num3(num3), Base (num1, num2)

    void ShowRecord ()
    {
        Base::ShowRecord ();
        cout << num3 << endl;
    }
}
```

```

void printRecord ()
{
    Base::printRecord ();
    cout << nums << endl;
}

```

```
};
```

```
main ()
```

```

{
    Base base (10, 20);
    base.Derived :: ShowRecord ();    // NOT OK
    base.Derived :: printRecord ();   // NOT OK
}

```

```
Base * ptrBase = new Base (10, 20);
```

```
ptrBase -> Derived :: printRecord ();    // NOT OK
```

```
ptrBase -> Derived :: ShowRecord ();    // NOT OK
```

```
Derived d1 (10, 20, 30);
```

```
Derived d2;
```

```
d2 = d1;    // Assignment
```

```
d2.printRecord ();    // OK
```

```
Derived d3 = d1;    // Initialization, OK but diff. concept
```

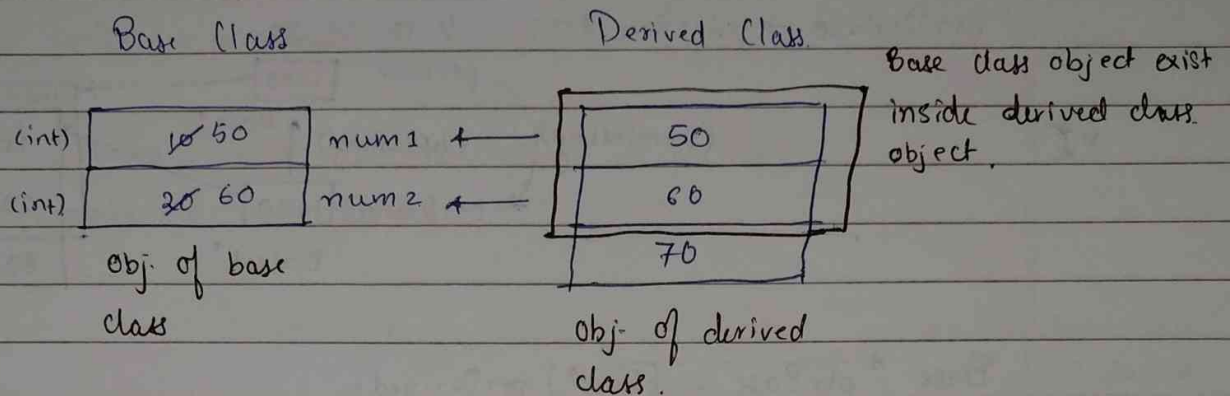
```
Base b1;
```

```
b1 = d1;    // assigning derived class obj. to base class obj.
```

```
// So, only members present in base class will get copied.
```

```
}
```

* Object Slicing



During object slicing mode of inheritance must be public.
(We must assign not initialize).
i.e.

```
Base b1 (10, 20);
Derived d1 (50, 60, 70);
b1 = d1; // Assignment
```

When we assign derived class object to base class object, only data members gets initialized in base class object, that are present in ~~base~~ class, (not other data members) derived.

* upcasting

```
Derived derived (50, 60, 70);
Derived *ptrDerived = &derived;
ptrDerived -> printRecord(); // Derived Printrecord.
```

OR

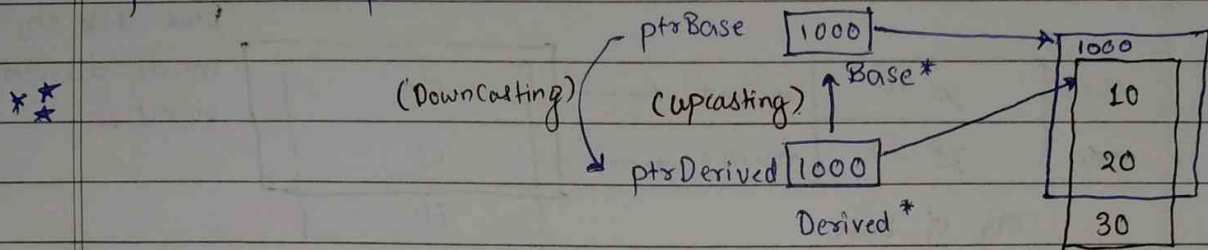
```
Base *ptrBase = (Base*) ptrDerived;
Base *ptrBase = ptrDerived;
```

} Same as typecasting is optional.

Teacher's Signature:

* Derived * ptrDerived = new Derived (10, 20, 30)

ptrDerived → printRecord(); // 10, 20, 30



Base * ptrBase = (Base*) ptrDerived;

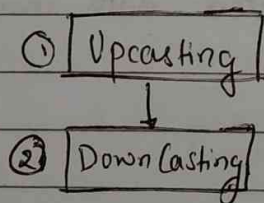
ptrBase → printRecord(); // 10, 20

pointer.

* Storing address of derived class in a pointer of base class, this concept is known as Upcasting. (Derived) → (Base)

* In upcasting, casting is optional, as every derived class is a base class.

*** Always do downcasting after doing Upcasting.



* Storing address of pointer of base class in a pointer of derived class, this concept is called Downcasting.

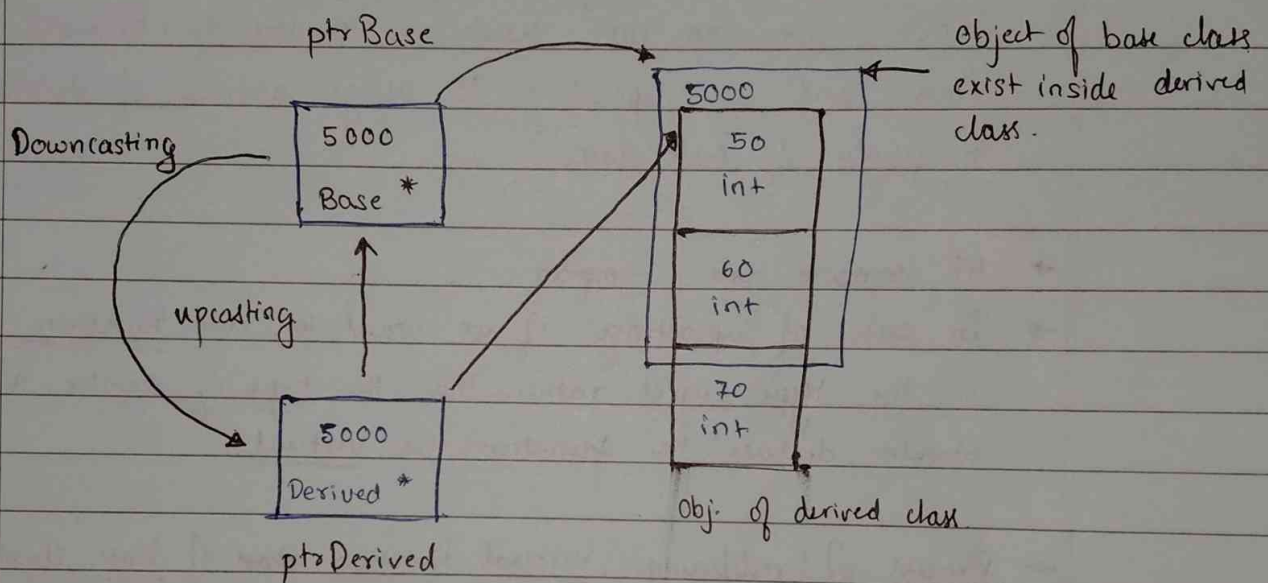
(Base) → (Derived)

* In downcasting, casting is compulsory.

In One line

```
Base *ptrBase = new Derived (10, 20, 30) // Upcasting  
ptrBase -> printRecord(); // 10 20
```

```
Derived *ptrDerived = (*Derived *) ptrBase;  
ptrDerived -> printRecord(); // 10 20 30
```



* During upcasting, if we want to access,

- ① Data member of derived class
- ② Non overridden function derived class.

then we should use down casting.

* What is Virtual Function?

→ In case of upcasting, a function which gets called ~~using~~ depending on type of object rather than type of pointer is called ~~a~~ virtual function.

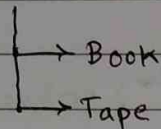
* We can't override non-virtual function & it should be present in Base class.

* If class contains any one virtual function, it's called

Polymorphic class.

* Upcasting is used to write maintainable code.

(Classes) Product



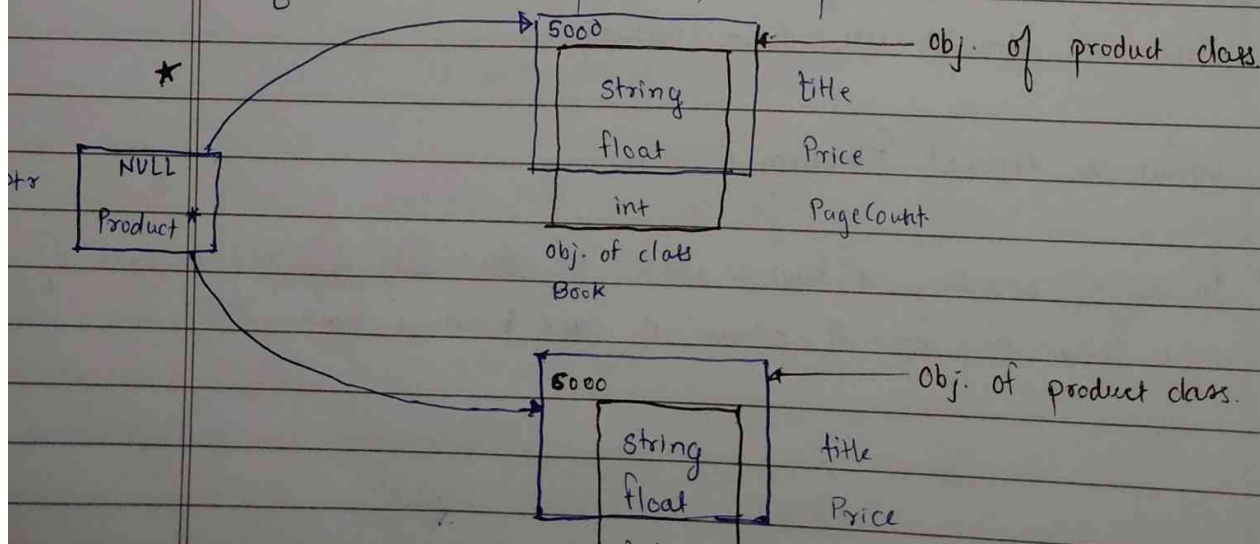
→ Here, instead of writing/creating objects for each class (Book or Tape), we can just create a pointer of (Product) Base class and use upcasting to assign address of derived class to pointer of Base class.

→ At compile time, compiler

→ In case of upcasting if we want to call function depending on the type object rather than the type of pointer then we should declare the function as virtual.

→ Process of redefining, virtual function ~~class~~ of base class, inside derived class with same signature is called function overriding.

→ We declare a function virtual, when the logical implementation of function is partial/incomplete.



* Pure Virtual Function

→ If the implementation of member function is 100% incomplete, then we should make that function pure virtual function.

- Pure virtual function does not have a body.

Syntax:-

```
virtual void method-name () = 0 ;
```

- If a class contains at least one pure virtual function, it is a abstract class.

- ~~In pure virtual function~~

- In derived class, we need to override pure virtual function compulsorily, otherwise it also becomes an abstract class.

- virtual void func () = 0; // OK

- virtual void func () = 0 {} // NOT OK

- void func (void) = 0 {} // NOT OK

- void func (void) = 0 // NOT OK

- In c++, a pure virtual function is also called as abstract method.

* If a class contains all pure virtual functions, then such class is called pure abstract class / interface.

* We cannot create object ^{of} ~~per~~ abstract class, but we can create a pointer of abstract class.

* We use downcasting when calling non-overridden member function.

Page No.:

Date: / /

```
class A
```

// Pure abstract class / Interface

```
{
```

```
    virtual void f1 () = 0;
```

```
    virtual void f2 () = 0;
```

```
}
```

// Interface Inheritance.

```
class B : public A
```

// Pure abstract class / Interface

```
{
```

```
    virtual void f3 () = 0;
```

```
}
```

* Run-time Polymorphism

Process of calling member function of derived class using pointer of Base class is called run-time polymorphism.

* Overriding virtual function is optional, but Overriding pure virtual function is mandatory.

Points To Remember (about if making a funⁿ virtual or not)

● If implementation of base class member function is

- ① 100% complete, then we should declare it non-virtual.
- ② partially complete/incomplete, make it virtual.
- ③ 100% incomplete, make it pure virtual function.

typeid $\approx$ instanceOf
(C++) (Java)

Page No.:	
Date:	/ /

* Why we can not declare virtual constructors?

- Virtual function is designed to call on base class pointer/reference
- We can not call constructor on object/pointer/reference explicitly. It's designed to call implicitly.
- Since we cannot call constructor on pointer/reference, we can't declare it virtual.

* Can we declare static member function virtual?

- Static member function are called using class name.
- But, virtual _{member} function is called using pointer/Reference.
- Since static member func. is not designed to call on base class pointer/reference, we can't declare it static.

* RTTI (Run-time type info)

- it's a process of getting the type of any object at runtime.
- to use RTTI, we should use typeid operator.

```
cout << typeid(number).name;
```

- typeid operator returns reference of constant object of type info class which contains type information.
- The class needs to be polymorphic, to get correct answer in/while doing upcasting, i.e. to get correct type of (ptrBase) & (*ptrBase).
- And to make the class polymorphic, we make the destructor virtual.

Teacher's Signature:

Day 6

- ★ While being in runtime, to get the type of object, we can use typeid to check for type & then call that ~~ex~~ class's member functions.
- ★ we check type & then do downcasting.
- ★ Downcasting is done in order to call non-overridden functions.

of.
~~calling~~

Virtual Destructor

- Make destructor of derived class virtual, while doing upcasting because, the ^{derived} class was not polymorphic, i.e. it sees the type of pointer and not the type of object.
- Due to early binding, when the object pointer of the base class is deleted, which was pointing to the object of derived class then, only the destructor of base class is called.
- It doesn't invoke the destructor of the derived class, which leads to the problem of memory leakage, in our program resulting in undefined behaviour.
- To correct this situation, ~~the~~ ^{the base} class should be defined with a virtual destructor.

```
virtual ~Base () {
```

```
    --  
}
```

* Early Binding / Late Binding

- virtual function call with pointer late binding

* Algorithm

if (function does not exist in caller data type)

Error :

else

{

if (caller is object)

Early Binding : caller type / inherited function will call

else

{

if (function is virtual)

Early Binding : caller type / inherited function will call

else

Late Binding : Object type / inherited function will call

}

}

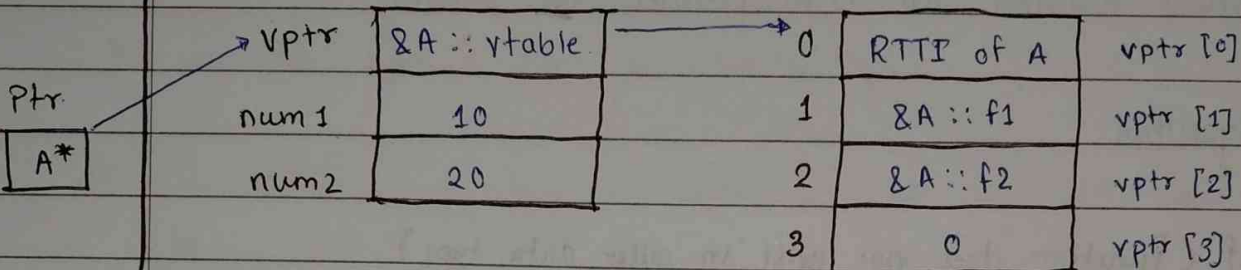
* Scenarios for late / Early Binding.

- ① If call to function gets resolved at compile time, then it's Early Binding
- ② If call to function gets resolved at run-time, then it's Late Binding
- ③ If we call any virtual / non-virtual function on object, then it's Early Binding.
- ④ If we call any non-virtual function on pointer / reference, then it's Early Binding.
- ⑤ If we call any virtual function on pointer / reference, then it's Late Binding.

Teacher's Signature:.....

* Virtual Function Table.

Stack



* If we declare a function virtual, compiler implicitly creates one table (array/structure) to store address of that virtual function. Such table is called of virtual function table.

* It's also called vt-table / v-table.

* Compiler generates v-table once per class.

* Virtual Function Pointer:- To store address of v-table, compiler implicitly declare one pointer as a data member inside class. and this data member is called vptr.

* vptr gets space once per object.

* Exception Handling

OS Resources :- memory, file, socket, thread, connection, system call.

4 types of error :-

- ① Compile-time
- ② Run-time
- ③ Logical
- ④ Linker

How to handle exception?

- ① Try
- ② Catch
- ③ throw

If matching catch block is not present for thrown exception, then it'll internally call `terminate()` & then `abort()` the function / flow of code.

```
try {
```

```
    // code code which might throw.
```

```
}
```

```
catch (type-of-exception e)
```

```
{
```

```
    cout << "Exception Caught" << endl;
```

```
}
```

* Runtime error is also called as exception.

* `break`, `return`, `goto`, `throw` are jump statements.

* If a function doesn't throw exception, we can specify it using `noexcept` keyword.

* `return_type function_name () throw (exception)`
Exception throw record list

* It's recommended to handle exceptions centrally, i.e. in main function.

- ① Try
- try is a keyword in C++;
 - try block is also called as try handler;
 - we cannot define try block after catch block;
 - there should be at least one catch block for a try block.

- ② Throw
- throw is a keyword in C++;
 - If we want to generate exception explicitly then we should use throw keyword.
 - "throw" statement is a jump statement.

- ③ Catch
- Catch is a keyword in C++;
 - catch block is also called as catch handler.
 - catch block is used to handle exceptions ~~for~~ thrown from try block.
 - Single try block can have multiple catch blocks.
 - cannot define catch before try block.
 - A catch block which can handle all type of exception is called a generic catch block.

```
( catch (...) {  
    // code  
}
```

Teacher's Signature:

* Template (C++)

Syntax:- `template < typename T >`

```
void swapping ( T &a , T &b)
{
    T temp = a ;
    a = b ;
    b = temp ;
}
```

`Swapping <int> (1, 2);` // Pass data type using angled brackets.
`Swapping <double> (1.0, 2.0);`

OR

`Swapping ("abc", "xyz");` // Type Inference.

(automatically assumes type of parameter)

- * Using template we can write generic code.
- * It is used to reduce developers effort.
- * Designed for implementing generic DSA.

Types:- ① functional

- Type inference
- can pass multiple type arguments to function.

② class

Teacher's Signature:.....

* Shallow Copy

- Process of copying contents of objects into another object as it is, is called shallow copy.
- Compiler by default creates a shallow copy.
- It's also called as bit-by-bit copy.

```
int num1 = 10;
```

```
int num2 = num1;    → shallow copy
```

```
Complex c1 (10, 20);
```

```
Complex c2 = c1;    → shallow copy.
```

* Copy Constructor

- Copy constructor by default creates a shallow copy.

Syntax :-

class-name (const class_name &object)

```
Ex:- Complex (const Complex &other)
{
    this->real = other.real;
    this->imag = other.imag;
}
```

} Shallow
Copy

★ Copy constructor gets called only on initialization.

i.e. `Complex c2 = c1;`

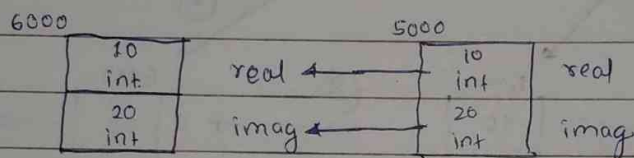
or

`Complex c2(c1);`

Dig:-

`int num1 = 10;`

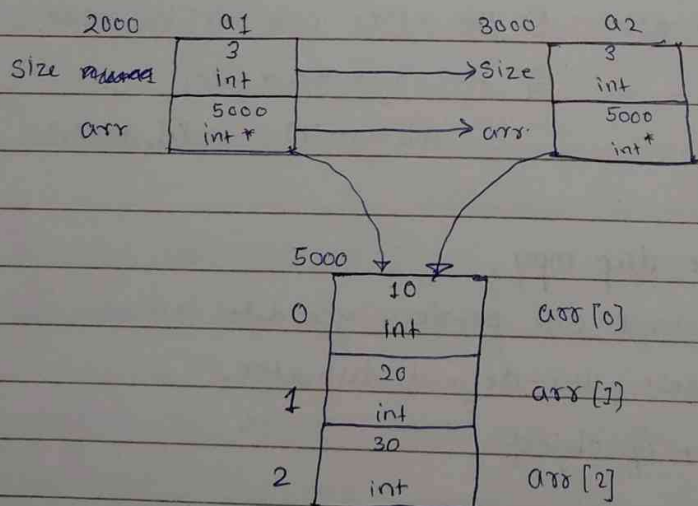
`int num2 = num1;`



`Complex c1(10, 20);`

`Complex c2 = c1;`

* Problem with Shallow Copy



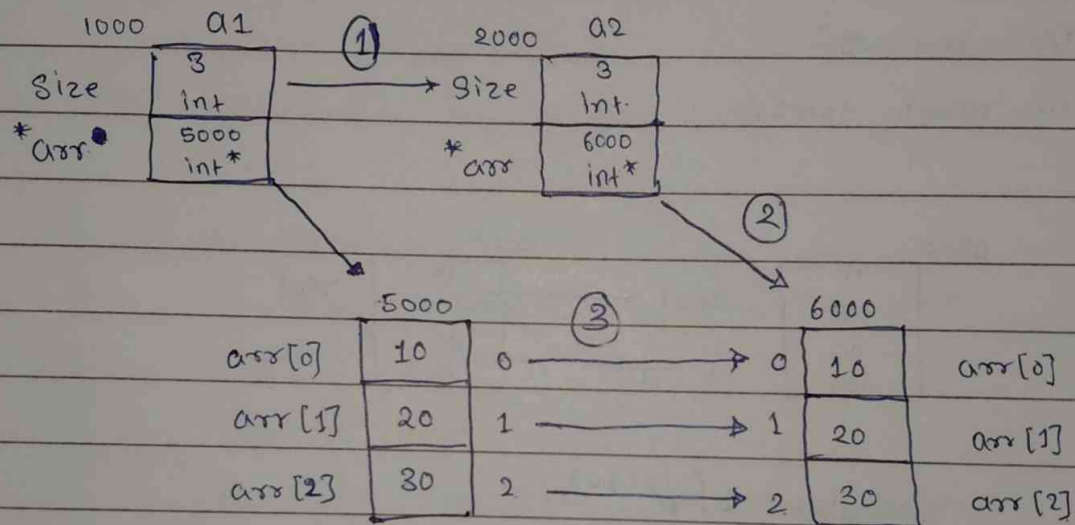
Dynamic array

★ Dangling pointer might get created.

Both the ^{pointer} objects are pointing to same memory address, which is not ideal.

Steps to create Deep copy.

- ① Copy the size
- ② Allocate memory for object destination object.
- ③ Copy contents from source obj. to destination obj.



Deep Copy / Memberwise Copy

```
Array a1 (3);    ① this->size = obj.size;  
Array a2 = a1;  ② this->arr = new arr[this->size];  
                ③ for (i=0 -> this->size)  
                    this->arr[i] = obj.arr[i];
```

Conditions to create deep copy.

- ① must contain at least one pointer type data member.
- ② class must contain user-defined destructor.
- ③ must create copy of object.

* Static-Cast :-

Static cast operator does not check if type conversion is valid or not valid, it just checks for the type of inheritance b/w source & destination.

Derived * derived = static_cast<Derived*>(ptrBase) // downcasting

Here, it does not check if we firstly do upcasting & then downcasting, it will give output, but even if we do directly downcasting it won't give error.

To do type conversion between compatible types we should use static-cast operator.

Static-cast is risky conversion, it should only be used when performance critical code when you are certain it won't fail.

Static-cast operator can be used to convert a pointer to base class to a pointer to derived class. Such conversions are always not safe.

* Dynamic-Cast :-

Dynamic cast operator checks if upcasting is done previously in case of downcasting, if upcasting is not done it will return NULL.

Dynamic cast operator check type conversion as well as the inheritance relationship b/w source & destination object.

To do dynamic cast, it should be polymorphic, and vice-versa.

* main function can be made friend function.

Page No.:

Date: / /

* Friend Function & Class

To access private, protected members of a class, ^{inside} ~~outside~~ ^{the} ~~a~~ ^{derived} class we can make that function a friend function.

Or we should declare derived class as a friend ~~class~~ in base class

```
class A {
```

```
    int a;
```

```
    friend void print();
```

```
};
```

```
    A a1;
```

```
void print() { cout << a << endl; }
```

```
int main ()
```

```
{
```

```
    A a1;
```

```
    :: print();
```

```
}
```

* We can make a class friend also.

```
friend class A;
```

* Friend function is a non-member function of the class that can access/modify the private members of the class.

* Friend functions are mostly used in operator overloading.

* Friend classes are used to implement data structures like linked lists.

* Dynamic Cast Operator

- in case of pointer if dynamic-cast operator fail ~~it returns~~ to do downcasting, then it returns NULL.
- in case of reference, if dynamic-cast operator fails to do downcasting then it throws bad cast Exception.

Day-7 * Standard Template Library

- collection of readymade template data structures & algorithms
- 4- Components
 - i) Containers
 - ii) Algorithms
 - iii) Functions
 - iv) Iterators
- Capacity of vector :- `v.capacity()`;
- Initially, when a vector is created, its capacity is 4.

Sequence Containers :-

- ① vector
- ② deque
- ③ list
- ④ forward list

Associative Containers :-

- ① set
- ② multiset
- ③ map
- ④ multimap

unordered_Associative Containers :- ① unordered_set

② unordered_multiset

③ unordered_map

④ unordered_multimap

Container Adapters :- ① Stack

② Queue

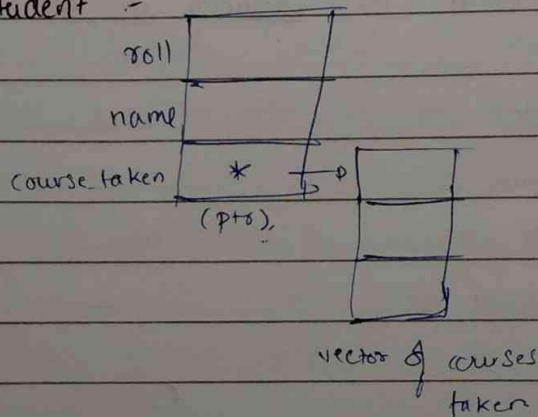
③ priority_queue

* Iterators :-

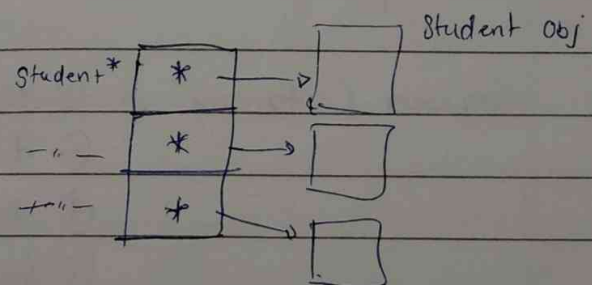
an object in C++ that enables the traversal of elements in a container (ex. vector, map, etc), provides access to elements

* Map :- used for Fast searching, Synt :- `map<int, int> mp;`

Student :-



Student-list :-



* Operator Overloading

- operator is taken in c++/c
- used to generate expression.

① Unary Operator

- requires only one operand
- ex:- (+, -, !, *) , &, !, ~, ++, sizeof, typed, etc.

② Binary Operator

- operator requires 2 operands.
- Ex:-
 - i) Arithmetic Operator
 - ii) Relational — " —
 - iii) Bitwise — " —
 - iv) Logical — " —
 - v) Assignment — " —

③ Ternary Operator

- Operator requires 3 operands
- Ex:- Conditional operator (? :)

→ If we want to use operator with objects of user defined type then we should overload operator.

→ To overload operator, we should ~~to~~ define ~~an~~ operator function.

- we can define operator function in 2 ways:-

① Non-member Function.

② Member function.

* operator function is called on left one.

*

Page No.:

Date: / /

* Complex c3: c1 + c2

↓

it resolves as c1.operator+ (c2)

- as operator+ is called on c1, so we should make operator+ a member function in Complex class

i.e.

```
Complex operator+ (Complex &other)
{
```

```
    Complex temp;
```

```
    temp.real = this->real + other.real;
```

```
    temp.imag = this->imag + other.imag;
```

```
    return temp;
```

```
}
```

≈ Similarly, for other binary ~~members~~ operators as well.

* In case of non-member function, we need to make it a friend function.

i.e.

```
class Complex
```

```
{ ---
```

```
---
```

operator+

```
friend Complex operator+(Complex &c1, Complex &c2);
```

```
}
```

operator+

```
Complex (Complex &c1, Complex &c2)
```

```
{ Complex temp;
```

```
    temp.real = c1.real + c2.real;
```

```
    temp.imag = c1.imag + c2.imag;
```

```
    return temp;
```

```
}
```

Teacher's Signature:.....

* For non-member function, the declaration ~~can be~~ of operator overloaded function

Ex:- Complex Operator+ (c1, c2)

Page No.:

Date: / /

* pre-increment & post-increment operator overloading

as member function:-

```
Complex c1 (10, 20);
```

```
Complex res = c1++; // c1.operator++ (0);
```

Operator overloading function:-

```
Complex operator++ ( int value Complex & other )  
{
```

```
    Complex temp;
```

```
    temp.real = this->real++; // 11
```

```
    temp.imag = this->imag++; // 21
```

```
    return temp;
```

```
}
```

as a non-member:-

```
Complex operator++ ( Complex & other, int value)
```

and make above function friend funⁿ in Complex class.

copy constructor of ostream class is protected, that's why when creating a copy of cout, take it as a reference.

Page No.:

Date: / /

* Advanced Operator Overloading

* insertion operator (<<)

Complex c1(10, 20)

cout << c1; // operator<<(^{cout}~~cout~~, c1);

→ void operator<< (cout, c1); // can overload only as a non-member function.

```
→ void operator<< (ostream &out, Complex &c1) {  
    out << c1.real << endl;  
    out << c1.imag << endl;  
}
```

* Extraction operator (>>)

but, if we do, cout << c1 << c2;

it will resolve as, operator<<(operator<<(cout, c1), c2);

```
ostream& operator<< (ostream& out, Complex& c)  
{
```

```
    out << c.real << endl;
```

```
    out << c.imag << endl;
```

```
    return out;
```

```
}
```

↓
need to return the reference of out in order to continuously print c1 & c2.

* Extraction Operator

```
Complex c1;
```

```
cin >> c1; // operator >> (cin, c1);
```

```
void operator >> (istream &in, Complex c)
```

```
{
```

```
    in >> c.real;
```

```
    in >> c.imag;
```

```
}
```

```
For, cin >> c1 >> c2;
```

```
istream& operator >> (istream &in, Complex &c)
```

```
{
```

```
    in >> c.real;
```

```
    in >> c.imag;
```

```
    return in;
```

```
}
```

* Assignment Operator Function.

```
Complex& operator = (const Complex &otherthis)
```

```
{
```

```
    this->real = other.real;
```

```
    this->imag = other.imag;
```

```
    return *this;
```

```
↑
```

```
(value at this)
```


* Subscript Operator overloading ([])

```
class Array
{
    private:
        int arr[3];

    public:
        Array (void) {
            arr[0] = 10;
            arr[1] = 20;
            arr[2] = 30;
        }
}
```

```
int operator [] (int index)
{
    if (index >= 0 && index < size)
        return arr[index];
    else
        throw Array Out Of Bound ("Invalid");
}
```

```
}
main ()
{
    Array a1;
    int element = arr a1[0];
    cout << element;
}
```

```
if main ()
{
    Array a2;
    a2[0] = 100; → for this to resolve, return reference
    int elem = a2[0] (i.e. int& operator [] (int index))
    cout << arr elem << endl; → 100
}
```

Teacher's Signature:.....

* Nested Class / Inner Class

- if nested class is private/
protected you can't make object outside the class.
- if nested class is public, you can create object outside the class.

```
Class Outer
```

```
{
```

```
    public:
```

```
        Class Inner
```

```
        {
```

```
        };
```

```
};
```

```
main ()
```

```
{
```

```
    Outer out; // outer class obj.
```

```
    Outer::Inner in; // inner class obj
```

```
}
```

- if inner class is private, we can make main, friend function of inner class.
- members ~~class~~ of inner class can't be accessed in outer class. & to access them in outer class, we need to make it friend class.

Day-8 * Conversion Function

- Converting fundamental to user-defined type.

```
Complex c1; 10, 20
```

```
int num = 10;
```

```
c1 = num; → Complex(int value);
```

```
Complex(int value)
```

```
{ this → real = value;
```

```
  this → imag = value;
```

```
}
```

} conversion function

* user-defined to fundamental

```
Complex c1(10, 20);
```

```
int num;
```

```
num = c1; → //c1.operator int()
```

```
func" :- operator int ()
```

```
{
```

```
  return this → real;
```

```
}
```

// has no return because it's a special function.

From happening explicitly

* to prevent such type conversion ↑ we can use "explicit" keyword before function definition.

C++ File IO

fstream → ifstream (to read)
→ ofstream (to write)

modes :- "w" - write

"r" - read

"app" - append

"trunc" - truncate

- Write mode :-

```
void write ( void )  
{  
    ofstream fout ( "file.txt", ios::app );  
    fout << " DAC Course" << endl;  
    fout.close();  
}
```

- read mode :-

```
{  
    void read ( void )  
    {  
        ifstream fin ( "file.txt" );  
        string line;  
        while ( !getline( fin, line ) )  
        {  
            cout << line;  
        }  
        fin.close();  
    }  
}
```

- * `stringstream` class object can give us splitted data, i.e. it'll split data/line on basis of a char/symbol.

```
stringstream data (line);
getline (data, id, ',');
getline (data, name, ',');
getline (data, salary, ',');
```

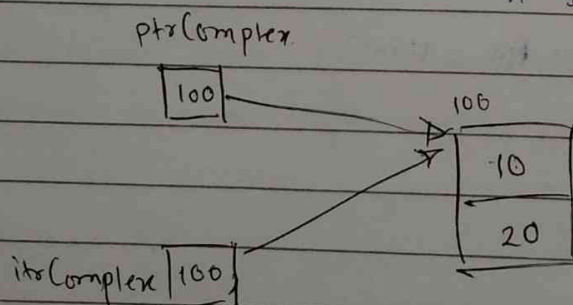
* Reinterpret Cast

- used to type cast pointer of any type to pointer of any other type.
- it's not recommended, but should be used safely.

```
Complex c1(10, 20)
Complex *ptrComplex = &c1;
```

```
int *itrComplex = reinterpret_cast<int*> (ptrComplex);
*itrComplex = 100;
itrComplex = itrComplex + 1;
*itrComplex = 200;
```

```
c1.printRecord(); // Real: 100
                  // Imag: 200
```



* Const Cast Operator

```
void displayRecord() const  
{
```

```
// Test * const ptr = (Test * const) this;
```

OR

```
Test* const ptr = const_cast<Test* const>(this);
```

```
ptr -> showRecord();
```

```
}
```

↓
non-constant member function

* Singleton Design Pattern

- create a class Singleton.

i) create a static Singleton pointer (data member private)

ii) make the constructor private.

iii) Initialize the static ptr to NULL (in global declaration)

iv) Create the static function that returns the instance of ^{this} class.

* Header guards are used to prevent multiple replacement of code from header files. (i.e. if we include header file multiple times, it'll only consider once)

```
#ifndef TIME_H
```

```
#define TIME_H
```

```
%header file code
```

```
#endif
```

Teacher's Signature:

* Const Cast Operator

```
void displayRecord() const
{
    // Test * const ptr = (Test * const) this;
    OR
    Test* const ptr = const_cast< Test* const > (this);
    ptr -> showRecord();
}
```

↓
non-constant member function

* Singleton Design Pattern

- create a class Singleton.
 - i) create a static Singleton pointer (data member private)
 - ii) make the constructor private.
 - iii) Initialize the static ptr to NULL (in global declaration)
 - iv) Create the static function that returns the instance of ^{this} class.

* Header guards are used to prevent multiple replacement of code from header files. (i.e. if we include header file multiple times, it'll only consider once)

```
#ifndef TIME_H
```

```
#define TIME_H
```

```
%header file code
```

```
#endif
```

* Interview Questions (OOPs)

* what are object-oriented concepts? what is diff b/w object-based and object-oriented and fully-object-oriented programming language?

- Abstraction
- Hierarchy → Reusability
 - is-a (inheritance)
 - has-a (association)
 - uses-a (dependency)
 - creates-a (instantiation)
- Modularity
 - Divide source code into multiple logical units - better maintainability

Minor Pillars :- May be supported by any OOP lang.

- Typing
- Concurrency
- Persistence

* what is abstraction & encapsulation?

Abstraction - getting essential thing from the system.

- Hides unnecessary details
- Describes outer behaviour of objects
- Implemented with abstract classes & interfaces.

Encapsulation - Binding code and data together in a class

- defines inner view of the system.
- it provides data-hiding

runtime polymorphism = dynamic method dispatch.

Page No.:

Date: / /

* Advantages of OOP language?

- Data Security
- Modularity (classes / namespaces)
- Code Reusability (Association & Inheritance)

* What is class & object?

- class is a blue-print of the object. class has fields & methods
- Object is an instance of class.
- class is a logical entity.
- Object is a physical entity.

* Characteristics of object?

- state
- behaviour
- identity

* Use of getter & setter methods? to provide access to private data members.

* What is polymorphism?

- Taking many forms
- Same name, diff. implementations

Types :- ① Compile time polymorphism.

② Run-time polymorphism.

- Function to be called is decided at run-time, then it's Late binding, depending on type of object.

Day-9

dll - dynamic linked library

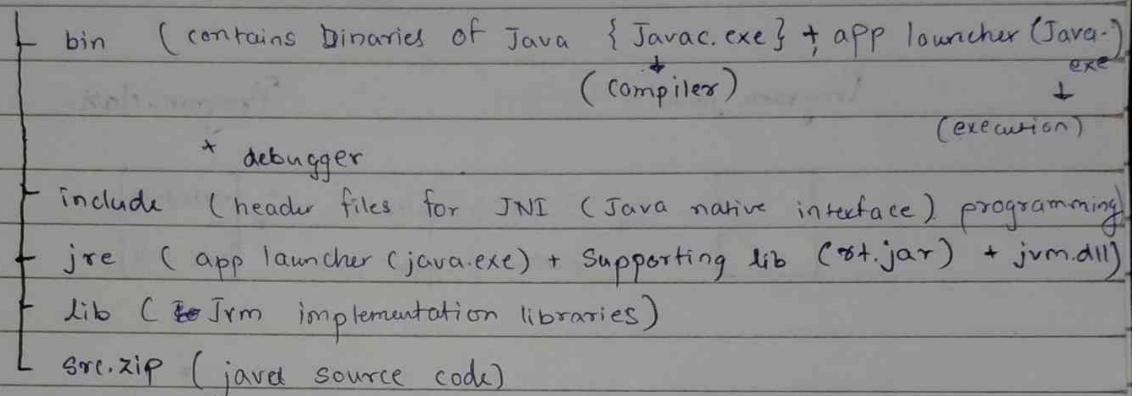
Core Java Basics

Page No.:

Date: / /

- Java Platforms :-
 - ① Java ME (micro edition) - (mobile apps)
 - ② Java SE (standard edition) - (Desktop apps)
 - ③ Java EE (Enterprise edition) - (Web Programming)
- File structure of Java 8 :-

Java



- In java, everything is written inside a class.

```
public class Program
{
    public static void main (String[] args)
    {
        System.out.println ("Hello");
    }
}
```

- Name of file and class should be same.

So, above file should be, (Program.java),

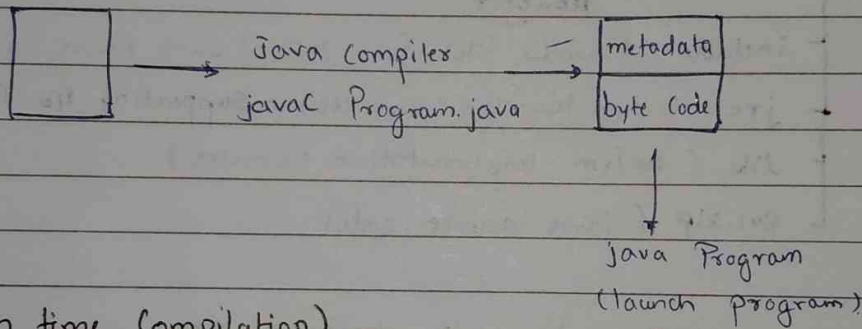
JavaC Program.java $\xrightarrow{\text{outputs}}$ Program.class
(compile command) (byte code)

Teacher's Signature:

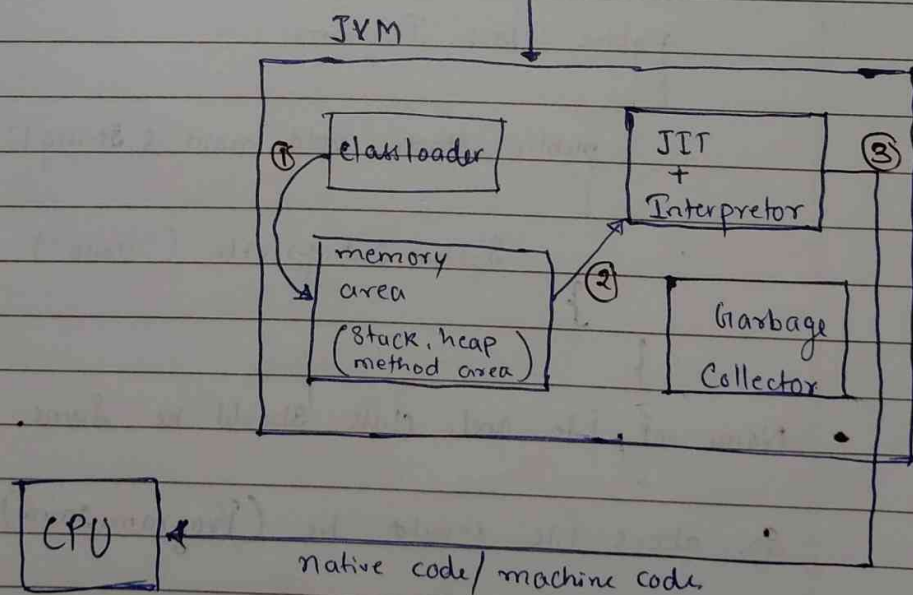
- Compiler ~~the~~ compiles the java source code into byte code
- Java Program → (Executes the code)
- * Java Virtual Machine :- this takes in bytecode (.class) and converts it into native (CPU understandable code) code.

Program.java

Program.class



(JIT :- Just in time compilation)



★ Every object in java is created on heap.

Page No.:	
Date:	/ /

- Class loader loads the class in memory area.
- JIT (Just in time compiler) :- it caches code, which might be repetitive so, interpreter doesn't interpret multiple times (reducing overhead)
- Garbage Collector:- used to release memory

* Development Tools :- Compiler + Assembler + Linker + Profiler + IDE
↓
(optional)

Documentation :- java docs, MSDN, MAN pages.

Supporting Libraries

Runtime Environment

All above 4 combined is called a SDK (Software development kit).

* Java's SDK = JDK → java docs + java dev tools + java supp. lib + Java Runtime env.

→ java docs + java dev tools + rt.jar + ~~JRE~~ JVM

→ java docs + java dev tools + JRE

JVM → used to run the code

rt.jar → contains precompiled library files.

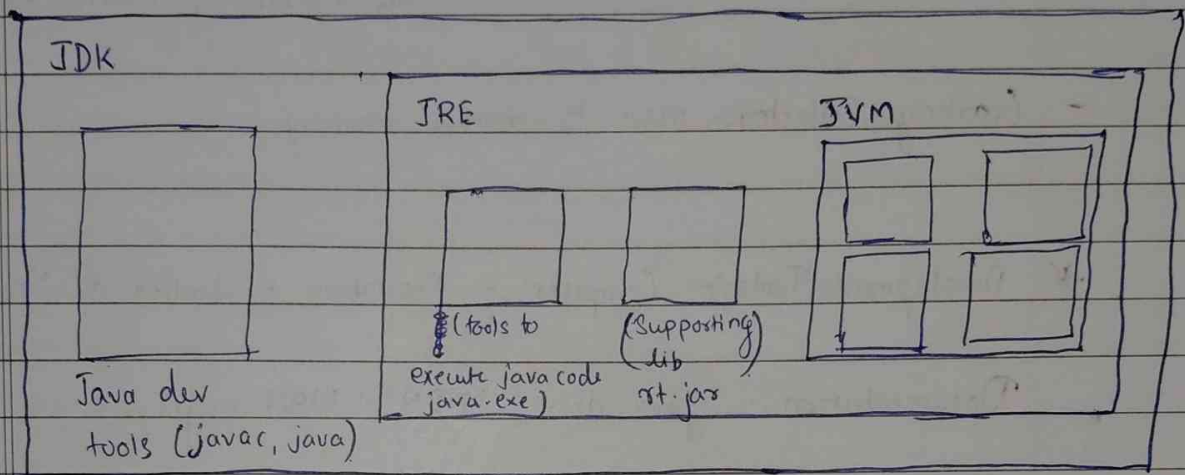
JRE = JVM + rt.jar

need

For developer :- We only JDK.

For client :- Client needs only JRE, to run his application.

(Imp)



Supporting lib (rt.jar) :- Frequently used classes.

* Java is dynamic, i.e. it loads only those classes which are used in code.

(Imp) * Single .java file can have multiple classes, and for each class a separate .class file is created. Each file has its own separate main method & we need to execute the .class file separately by giving separate java cmd. (All classes are not made public)

(Imp) * Single .java file can have multiple classes, but only single public class and the name of file should be same as public class.

(Imp) * Multiple public classes can be written in multiple .java files.

Teacher's Signature:

* (Spring Tool Suite .exe) → SDE

Page No.:	
Date:	/ /

This class should be accessible outside package

```
* public class Program
{
    This method should be accessible outside class
    public static void main (String[] args)
    {
        System.out.println ("Hello");
    }
}
```

Static :- main method is called without creating the object of class, static belongs to class (it's called on classname).

Void :- main method does not return anything

main :- Entry point method.

- program execution begins from main

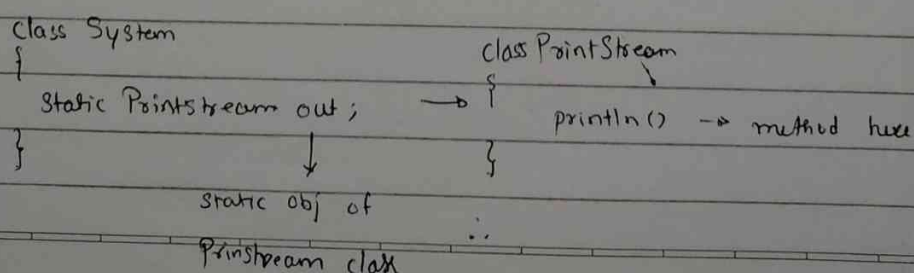
String[] args :- Command line argument

- we can pass argument on command line.
- ex:- java Program arg1 arg2 arg3

System :- it's a class in java.lang (package) { java.lang.System }

System.out :- it will print the o/p on console / terminal.

println() :- It's a method of printStream class which is used to print message / value.



* Java is object oriented language.

1) Class

- logical entity / blueprint
- Similar to structure in c
- methods & fields

2) Objects

- physical entity
- Instance
- Object will get space on heap section.

* Classpath :- It is set of directories separated by ';' (windows) & ':' (linux).

- It helps java tools (compiler, launcher) to locate java classes and packages.
- java tools searches for packages / classes in given classpath

* * - if classpath is not set it will search in curr. directory.

* `javac -d .\bin Program.java`

↓
destination
Folder

↓
creates a
Folder &

stores .class file generated from compilation.

* To set classpath :- set `CLASSPATH = .\bin ;` → (non-persistent)

- main method can be overloaded in java, in same scope, but JVM identifies only (public static void main (String[] args))
- In java in same scope we can give same name to method but different argument.
- it's also called as method overloading.

```
public class Program
{
    public static void main (String[] args)
    {
        System.out.println ("Hello");
        Program.main (10, 20);
        Program.main ("10", "20");
    }
}
```

```
public static void main (int a, int b)
{
    System.out.println ("2 int arg");
}
```

```
public static void main (double a, double b)
{
    System.out.println ("2 double arg");
}
```

* Java is strictly typed language.

* Data types - ① Primitive types or Value types
② Non-Primitive types or Reference types

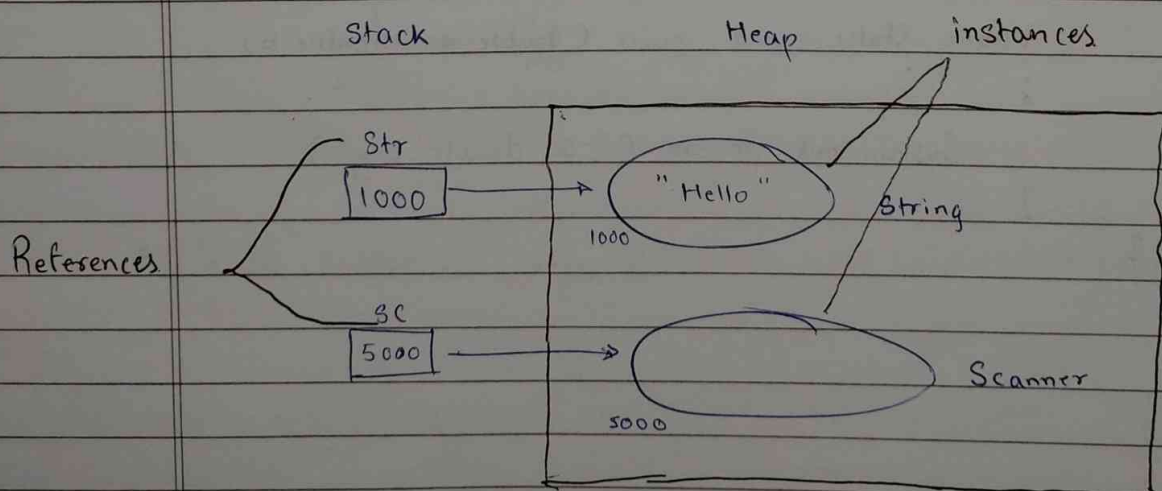
Primitive types

- Boolean - boolean
- Character - char
- Integral - int, short, long, byte
- Floating Point - float, double

Non-Primitive types (Reference types)

- Class String str = "Hello";
- Interface
- Enum Scanner sc = new Scanner (...);
- Array reference instance

* things on left are stored in stack & things on right are stored in Heap.



Teacher's Signature:.....

* References are immutable.

String str; → will only create reference

* byte	1 byte	float	4 byte
int	4 byte	double	8 byte
short	2 byte	long	8 byte
char	2 byte	boolean	1 byte

* In java, you can't keep values uninitialized.

to output values :-

```
System.out.println("Area:" + area); // Area:10
```

* Scanner class

- used to take input from user / keyboard.

```
Scanner sc = new Scanner(System.in);
```

```
String name = sc.next(); // will take ip from keyboard & store in name
```

```
int roll = sc.nextInt(); // will take int as input
```

```
double marks = sc.nextDouble(); // will take double as input.
```

- to scan multiple words we can use "nextLine()" method.

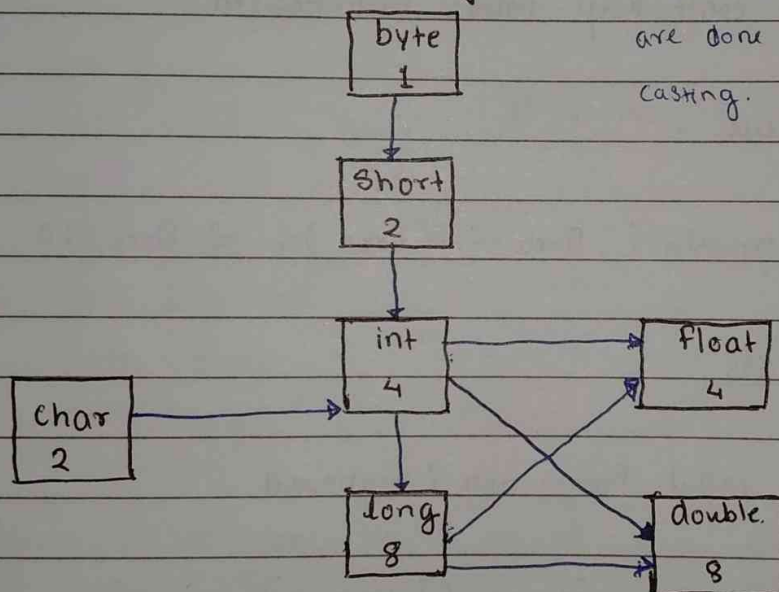
```
String FullName = sc.nextLine();
```


* `print()` → same line `println()` → new line

* when taking int input before string input using `nextLine()`, when we input number & press enter (`\n`), (`\n`) gets into `nextLine` and gets executed, skipping the input.

To avoid this, we can just add one more `nextLine()` before it;

* Narrowing and Widening



* conversions shown by arrow are done directly, no need for casting.

larger to smaller → narrowing (explicit casting required)

smaller to larger → widening

* `short s = 1;`

* `int i = 4;`

`int i = s;` // widening

`short s = (short) i;` // narrowing